AEGISPAY — SELL

LangGraph AI-buyer checkout · V3

A safe end-to-end purchase where the AI helps but never controls money. Payment finality comes from the provider.

The flow

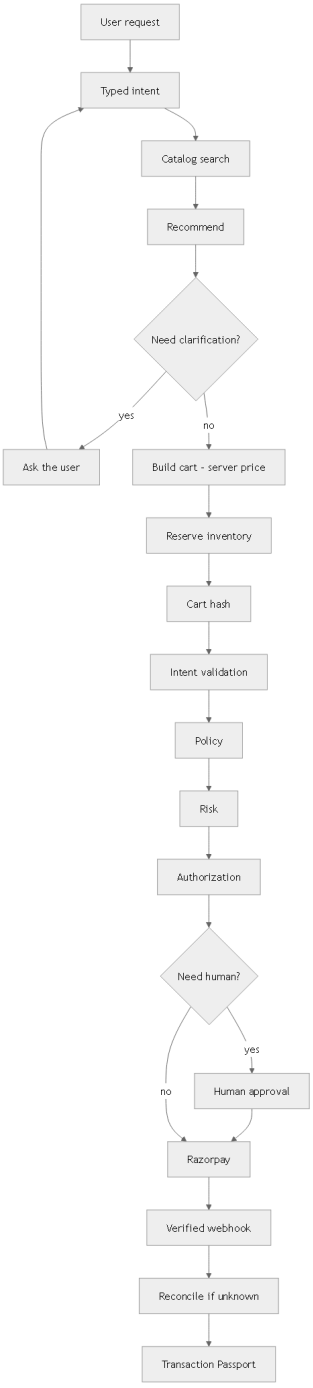

Figure 1 — From a user request to a verified Passport

1 **User request.** “Find running shoes under ■4,000.”

2 **Typed intent.** The AI's output is a strict, validated schema.

3 **Catalog search.** A real, parameterized query.

4 **Recommend.** Best matches, priced by the store.

5 **Clarify if needed.** Ask, then continue.

6 **Build cart.** Server-owned price.

7 **Reserve inventory.** Stock held so it can't oversell.

8 **Cart hash.** A change invalidates authorization.

9 **Validate intent.** Strictly checked.

10 **Policy.** Limits, categories, hours.

11 **Risk.** Explainable score.

12 **Authorization.** Scoped, expiring, single-use.

13 **Human if required.** High value → a person approves.

14 **Razorpay.** Only approved money moves.

15 **Verified webhook.** Provider truth.

16 **Reconcile if unknown.** Never a blind retry.

17 **Transaction Passport.** Full verifiable proof.

Explicit failure paths

Failure	Safe state	Recovery
Price changed	authorization invalid	revalidate cart, ask consent again
Inventory expired	cart not locked	re-select / notify
Approval expired	no action, expires	re-request approval
Payment failed	PAYMENT_FAILED	order failed safely; retry is a new attempt
Payment UNKNOWN	UNKNOWN, first-class	reconcile → complete / fail; never blind retry
Webhook duplicate	safe no-op	acknowledge only
Refund	gated by policy + authorization	idempotent, capped to captured

The payment state machine (simplified)

CREATED → **CART_LOCKED** → **AUTHORIZATION_PENDING** → **AUTHORIZED** → **PAYMENT_PENDING** → **PAID** → **ORDER_CONFIRMED**, with failure states AUTHORIZATION_EXPIRED, PRICE_CHANGED, INVENTORY_EXPIRED, PAYMENT_FAILED, PAYMENT_UNKNOWN, ORDER_FAILED, REFUND_PENDING → REFUNDED, and CANCELLED.

- **UNKNOWN is first-class.** It exits only on a verified webhook or reconciliation result.
- **Never blindly retry UNKNOWN.** That is how duplicate charges happen.
- **Provider truth.** The frontend and the agent never set payment state.

Key protections

- Typed intent · server-owned prices · cart hash · price version · inventory reservation · cart expiry · idempotency (designed to prevent duplicate financial effects) · scoped/expiring authorization · agent identity · prompt-injection defense · refund/cancellation controlled and audited.
- Product ownership lock: a product is bound to one merchant (tenant) with a per-merchant unique SKU. It can only be added to a cart whose merchant matches — two merchants cannot share or add the same product, and a cross-merchant product is rejected.

Why AegisPay is different

AI can reason and recommend, but only the deterministic AegisPay control plane validates, authorizes and controls financial actions. Razorpay executes only approved actions.

- **No direct money path.** The AI runtime has no payment keys, no DB credentials, no unrestricted money tools.
- **Deterministic authorization.** Policy, risk and a scoped, expiring authorization gate every action.
- **Provider truth.** Payment finality comes only from a verified webhook or reconciliation — never a guess.
- **Prevents, not promises.** Idempotency + UNKNOWN-never-blind-retry are designed to prevent duplicate financial effects.
- **Tamper-evident.** Every decision is recorded in a hash-chained, anchored audit ledger and Transaction Passport.
- **Simple to run.** Two services, PostgreSQL + Redis + SQS on ECS — no unnecessary complexity.